

When DI Becomes the Problem

INJECTION EXPOSES DESIGN PROBLEMS – IT DOES NOT AUTOMATICALLY SOLVE THEM

```
class CheckoutService {  
    CheckoutService(  
        CartRepository carts,  
        PricingService pricing,  
        InventoryService inventory,  
        PaymentGateway payments,  
        FraudChecker fraud,  
        ShippingService shipping,  
        EmailSender email,  
        Analytics analytics  
    ) { /* DI did not make this cohesive */ }  
}
```

The constructor is giving useful feedback: this class probably owns too many responsibilities. Hiding those dependencies behind a container would only hide the smell.

Injecting everything

Values and tiny pure helpers do not need an interface, factory, and registration “just in case.” Add seams where change or testing needs them.

Constructor over-injection

A long parameter list is usually a cohesion problem. Split the class around capabilities or use-cases instead of bundling dependencies.

Service locator

Calling `container.get()` inside business code hides required dependencies and moves failures to runtime.

One interface per class

An interface with one permanent implementation can be ceremony without a boundary. Abstract behavior, not every concrete type.

The real test

DI should make boundaries clearer. If the wiring grows faster than the behavior, revisit the design—not merely the container configuration.